Prototype-Initialized Adaptation of ViT-B/16 for 10,000-Class Fine-Grained Classification under a Fixed Compute Budget

Member 1 (student ID) Member 2 (student ID)
Member 3 (student ID) Member 4 (student ID)
CS604 Team 06, Singapore Management University
email addresses

Abstract

We adapt ViT-B/16 (pretrained on ImageNet-21k, fine-tuned on ImageNet-1k) to a fine-grained dataset with 10,000 classes and only 45 training images per class. Each training run may use 2 hours on 48 GB of GPU memory. We compare seven ways to adapt the model under the same settings: a linear probe, fine-tuning the last few blocks, full fine-tuning, BitFit, Adapter, AdaptFormer and LoRA. The choice that mattered most was how to start the new 10,000-class head. The usual start, a linear probe trained on frozen features of the training images, made all three methods we tested worse. Starting the head from the mean feature of each class (a prototype) raised holdout accuracy by 2.1–2.5 points for full fine-tuning, LoRA and AdaptFormer, and raised the validation accuracy of our final model by 3.1 points. We also find that where the trainable parameters are matters more than how many there are: LoRA on 1.375% of the backbone is as accurate as fine-tuning the last four blocks (33%). Training memory depends on how far back the gradients go, not on how many parameters are trained. Our final full fine-tuning model, trained on all images and tested with horizontal flips, reaches 76.22% top-1 on the CodaBench validation set (reference: 70.8%) and 76.09% on the test set, in 1.03 GPU-hours on one A100. Our final LoRA model trains 1.375% of the backbone and reaches 71.90% on validation (reference LoRA: 68.3%) and 71.67% on test; after training it is merged into the weights, so it does not slow down inference. We also report the ideas that did not work, and show after the deadline that larger input images add another 4.4 points on our holdout.

1. Introduction

Large pretrained Vision Transformers [5, 19] are usually not trained from scratch. Instead, they are adapted to a new task, either by fine-tuning all weights or by training a small number of added parameters [3, 6, 7]. In this project we adapt ViT-B/16 to a fine-grained classification dataset with 10,000 classes whose names are hidden. The training set has 450,000 images, exactly 45 per class. The validation set (50,000 images, 5 per class) and the test set (100,000 images, 10 per class) have hidden labels and are scored on CodaBench. Each training run may use at most 2 hours on 48 GB of GPU memory (96 GB·h), and for every method we report four numbers: top-1 accuracy, trainable parameters, inference latency and training GPU-hours.

Two things make this task different from common transfer-learning benchmarks. First, the new classifier head is large, but each class has few images. A $10{,}000 \times 768$ linear head has 7.69M parameters: 9.0% of the 85.8M-parameter backbone and 26× more than a rank-8 LoRA on the attention layers. Yet each row of the head is trained on only 43–45 images. Second, the budget is tight. One epoch of full fine-tuning takes about 7 minutes on an A100, so only about eight epochs fit into one run.

This leads to three questions. (1) How should we start the new head? (2) Which adaptation method should we use, and where should its trainable parameters go? (3) How should we split the budget between epochs, data and test-time computation? To answer them we ran 26 runs, changing one setting at a time. We choose models on a local holdout split, whose accuracy is within 0.3 points of the CodaBench validation score. Our contributions are:

- **A better start for the head.** The common linear-probe (LP) head, which LP-FT [12] trains on frozen features before fine-tuning, hurts in our task. Starting from class-mean prototypes [17] instead adds 2.1–3.1 points for full fine-tuning, LoRA and AdaptFormer. A random head does as well as the prototype head, so the harm comes from fitting the head on the training images (Sec. 4.6).
- **Seven adaptation methods under one protocol.** Small changes in every block give much more accuracy per parameter than fine-tuning the top blocks: LoRA on all linear layers (1.375% of the backbone) matches the last four blocks (33%), and BitFit (0.12%) beats the last block

(8.3%). LoRA is the most accurate of the efficient methods (98% of full fine-tuning), but only 0.44 points ahead of the Adapter; its real advantage is that it can be merged into the weights, so it adds no latency, while the two adapters add 33–40%. Training memory depends on how far back the gradients go, not on how many parameters are trained, so an efficient method can still need a lot of memory (Sec. 4.5).

- **Full cost reporting.** For every run we report accuracy, trainable parameters, latency, GPU-hours and peak memory. Our final models reach 76.22% (full fine-tuning, +5.4 over the reference) and 71.90% (LoRA, +3.6 over the reference LoRA) on the CodaBench validation set, and 76.09% and 71.67% on the test set.
- **The whole process.** We describe the study round by round, including the ideas that failed (Sec. 4.2). After the deadline we found that larger input images add another 4.4 points (Sec. 4.9).

2. Related Work

Vision Transformers and transfer learning. ViT [5] applies a Transformer [22] to a sequence of 16×16 image patches. Supervised pretraining on ImageNet-21k [4] with strong augmentation and regularization (AugReg [19]) gives backbones that transfer well. We use the AugReg ViT-B/16 checkpoint from timm [23] that was also fine-tuned on ImageNet-1k. Models that are better on ImageNet also transfer better, both with a linear probe and with fine-tuning [10]. Common fine-tuning recipes use AdamW [15], warmup with cosine decay [14], label smoothing [20], stochastic depth [8] and layer-wise learning-rate decay [1]; our full fine-tuning recipe uses all of them.

Parameter-efficient fine-tuning (PEFT). Adapters add a small bottleneck MLP to every Transformer block [6]. AdaptFormer [3] instead puts a scaled bottleneck branch *next to* the MLP of each ViT block. LoRA [7] learns a low-rank update $\Delta W = BA$ for frozen weight matrices, and this update can be merged into the weights after training. VPT [9] adds learnable prompt tokens to the input, and BitFit [2] trains only the bias terms. An older option is to fine-tune only the top layers, whose features are the most task-specific [24]. We implement LoRA with the PEFT library [16] and compare it with Adapter, AdaptFormer, BitFit and top-block fine-tuning under one protocol.

Starting the classifier head. When a pretrained model is moved to new classes, the old head is replaced by a new one; ViT starts it at zero [5]. LP-FT [12] argues that a random head distorts the pretrained features during fine-tuning. It therefore first trains a linear probe on frozen features and then fine-tunes everything. Prototype-based classifiers represent each class by the mean feature of its examples, as in prototypical networks [18]. These normalized class means can be copied (*imprinted*) directly into the weights of a cosine classifier [17]. Our prototype head is such an imprinted head with a chosen temperature. We compare it with a random head and a linear-probe head in a setting with many classes (10,000), few images per class (45) and a short training budget.

3. Method

Figure 1 shows the pipeline. Let f_θ map an image x to its final-layer `[CLS]` feature $z = f_\theta(x) \in \mathbb{R}^{768}$, and let $h(z) = Wz + b$ with $W \in \mathbb{R}^{10000 \times 768}$ be the new classifier head.

3.1. Backbone, budget and full fine-tuning

Backbone. We use the timm [23] checkpoint `vit_base_patch16_224.augreg2_in21k_ft_in1k`, the backbone that the project description recommends, and keep it in every experiment. It is ViT-B/16 pretrained on ImageNet-21k with AugReg [19] and fine-tuned on ImageNet-1k. The backbone has 85,798,656 parameters; we replace its 1,000-class head with a new 10,000-class head of 7,690,000 parameters. The project description also lists ViT-B/16 models trained only on ImageNet-21k. After the deadline we tested two of them with frozen features on our holdout (prototype head without training / linear probe): `augreg_in21k` reaches 49.96 / 61.05% and `orig_in21k`, the original Google checkpoint, 42.43 / 51.07%, against 43.95 / 61.94% for our backbone. Neither is better after a linear probe, so switching would not have helped.

Compute budget. The project allows $2\,\text{h} \times 48\,\text{GB} = 96\,\text{GB·h}$ per training run ($\pm 10\%$). On our 80 GB A100 this is 1.2 hours. Our clock counts all GPU work for training: head initialization (including feature extraction), all optimizer steps, and the holdout check after every epoch. It does not count unpacking the data, saving checkpoints or making the final predictions. Our code times training steps 20–120; if the planned epochs would use more than 97% of the budget, it shortens the cosine schedule, and it stops training at $1.05\times$ the budget. Every run in our results finished within 1.06 GPU-hours; the only longer run is one post-deadline test that is not part of our results (Sec. 4.9).

Full fine-tuning. All backbone and head weights are trained with AdamW [15] at learning rate 10^{-4} and weight decay 0.05 (none on biases, normalization parameters, the class token and position embeddings). We use layer-wise learning-rate decay of 0.75 [1] and drop path 0.1 [8]. The learning rate warms up linearly (one epoch for runs of 6 or

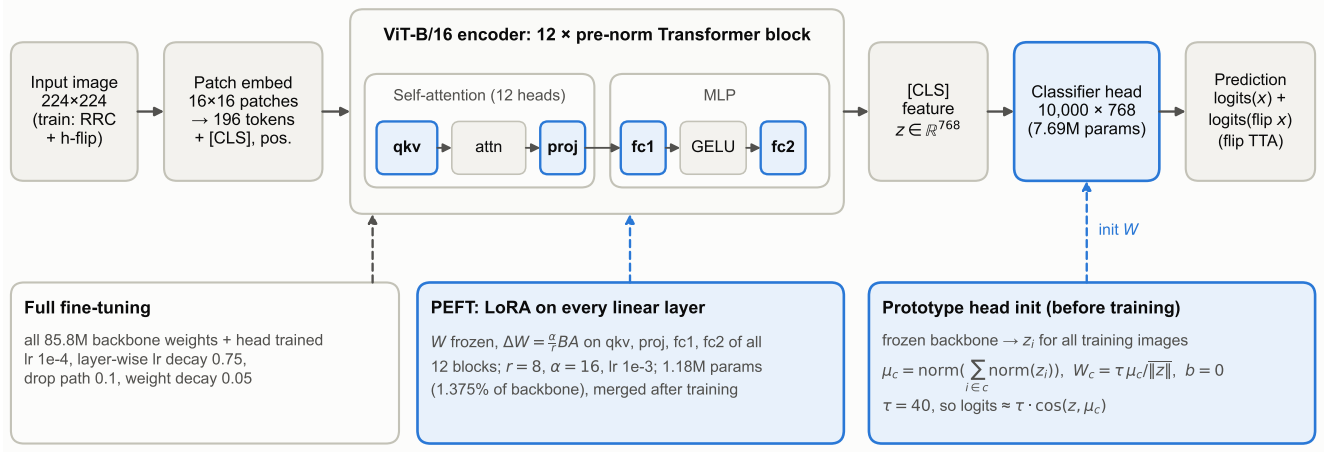


Figure 1. **Overview of our pipeline.** A ViT-B/16 backbone is adapted either by full fine-tuning or by LoRA on every linear layer (blue modules: qkv, proj, fc1, fc2 in all 12 blocks). Before training, the 10,000-class head is set to the L2-normalized class-mean features (prototypes), scaled by a temperature τ. At test time, the logits of the image and of its horizontal flip are added. All runs use AdamW with warmup and cosine decay, bf16 and batch size 256, within a budget of 96 GB·h (1.2 h on one A100-80GB).

more epochs, 0.3 epochs for 3-epoch runs) and then follows a cosine decay to zero [14]. Other settings are batch size 256, bf16 mixed precision, gradient clipping at norm 1, and label smoothing 0.1 [20]. Training augmentation is RandomResizedCrop (scale 0.25–1) plus horizontal flip. For evaluation, images are resized to 248 pixels and center-cropped to 224.

3.2. Parameter-efficient and partial adaptation

LoRA. For a frozen linear layer $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA [7] computes

$$y = W_0 x + \frac{\alpha}{r} BAx, \tag{1}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trained. B starts at zero, so training starts exactly from the pretrained model. We use $r = 8$ and $\alpha = 16$. timm joins the query, key and value projections into one `qkv` layer. So LoRA on `qkv` adapts all three, with the same parameter count (294,912, 0.344% of the backbone) as LoRA on separate query and value layers. Our final LoRA model applies LoRA to all four linear layers of every block (`qkv`, `proj`, `fc1`, `fc2`), for 1,179,648 parameters (1.375%). The LoRA weights and the head use learning rate 10^{-3} and weight decay 10^{-4}, with no LoRA dropout. After training, $\frac{\alpha}{r} BA$ is added into W_0, so inference uses exactly the base model.

AdaptFormer. Following [3], each block gets a bottleneck branch next to its MLP:

$$x_{l+1} = x' + \mathrm{MLP}(\mathrm{LN}(x')) + s \cdot W_{\text{up}}\, \mathrm{ReLU}(W_{\text{down}} x'), \tag{2}$$

where x' is the output of the attention sub-block. We use bottleneck width 64, scale $s = 0.1$, dropout 0.1 and a zero-initialized W_{up}, for 1,189,632 parameters (1.387%). Unlike LoRA, the branch cannot be merged into the backbone.

Method	Count	Parameters	%
Partial, last K blocks	$7{,}087{,}872\,K + 1{,}536$	7.09M–42.5M	8.26–49.6
Full fine-tuning	all	85,798,656	100
BitFit	$12 \cdot 8{,}448 + 1{,}536$	102,912	0.120
Adapter, AdaptFormer	$12 \cdot 99{,}136$	1,189,632	1.387
LoRA, qkv only	$12 \cdot 3{,}072\,r$	294,912	0.344
LoRA, all linear	$12 \cdot 12{,}288\,r$	1,179,648	1.375

Table 1. **Trainable backbone parameters** (head excluded; $r = 8$; partial: $K \in \{1, 2, 4, 6\}$). 1,536: final LayerNorm (partial) or the patch-embedding and final-norm biases (BitFit); 8,448 biases per block. Adapter: $2 \cdot 768 \cdot 64 + 64 + 768 = 99{,}136$ per block. LoRA adds $r\,(d_{\text{in}} + d_{\text{out}})$ per layer, *i.e.* $3{,}072\,r$ for qkv and $12{,}288\,r$ for qkv, proj, fc1 and fc2 together.

Adapter, BitFit and partial fine-tuning. For the comparison in Sec. 4.5 we add three more methods. The serial Adapter [6] applies the same bottleneck to the MLP output, $h + W_{\text{up}}\,\mathrm{ReLU}(W_{\text{down}} h)$ with $h = \mathrm{MLP}(\mathrm{LN}(x'))$, before the residual addition and without a scale. BitFit [2] trains only the bias vectors of all linear and normalization layers. Partial fine-tuning trains the last K blocks and the final normalization layer and freezes everything below them; one block has 7,087,872 parameters (8.26% of the backbone). Table 1 gives the trainable parameters of every method.

In all runs except the zero-shot baseline the head is also trained. The head alone has 7.69M parameters, so we report trainable parameters both without it (as in the project's 0.34% LoRA reference) and with it.

3.3. Starting the 10,000-class head

The head is the largest new part of the model, and each of its rows is trained on at most 45 images. A random head

gives almost uniform logits (initial loss $\approx \ln 10000 = 9.2$), so early in training the backbone gets large, noisy gradients from it. We compare three ways to start the head.

Random. timm's default initialization of a new linear layer.

Prototype (imprinted) head. With the frozen pretrained backbone, we extract the features z_i of all training images once. We then copy the L2-normalized class means into the head [17, 18]:

$$\mu_c = \frac{\sum_{i:\, y_i=c} \hat{z}_i}{\left\| \sum_{i:\, y_i=c} \hat{z}_i \right\|}, \quad W_c = \frac{\tau}{\overline{\|z\|}}\, \mu_c, \quad b_c = 0, \qquad (3)$$

where $\hat{z}_i = z_i / \|z_i\|$ and $\overline{\|z\|}$ is the mean feature norm, so each logit is about $\tau \cos(z, \mu_c)$. We pick the temperature τ from $\{5, 10, 15, 20, 30, 40, 60, 80\}$ by the cross-entropy on 10,000 training features, which gives $\tau = 40$. Without any training, this head already reaches 44.0% holdout accuracy. Feature extraction takes 6–7 minutes on an A100, and we count this time in every run's GPU-hours.

Linear-probe (LP) head. Starting from the prototype head, we train (W, b) on the cached frozen features for 10 epochs. We use AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 4,096, cosine decay and label smoothing 0.1. This reaches 62.1% holdout accuracy in a few seconds and is the first stage of LP-FT [12]. It was our first design, chosen so that a random head would not distort the features. Section 4.6 shows that it is the worst of the three.

3.4. Test-time augmentation

As is common [11], we add the logits of an image and of its horizontal flip. This doubles the inference cost, and we use it only where stated.

4. Experiments

4.1. Setup

Data and protocol. Validation and test labels are hidden, so we hold out 2 images per class from the training set (20,000 images, fixed seed). We use this holdout to choose models and run ablations, and train on the other 43 images per class. The final full fine-tuning model is then retrained on all 45 images per class; the final LoRA model keeps the holdout split and is trained on 43. Validation and test images are used only to make submission files. CodaBench reports top-1 accuracy; the validation set is balanced (5 images per class), so this is also the mean per-class accuracy. Our holdout is close to the server score: 72.31% *vs.* 72.57% for full fine-tuning and 71.98% *vs.* 71.90% for LoRA. At about 72% accuracy on 20,000 images, the standard error is $\sqrt{0.72 \cdot 0.28 / 20{,}000} \approx 0.32$ points, so we treat differences below about 0.3 points as noise. All runs use one seed.

Date	Question → experiment	Finding
Sep 28	Is our pipeline right? → reproduce the three references	all within 0.3 points (Tab. 3)
Sep 28	Do longer runs help? → full FT for 8 epochs, LoRA for 10	full +1.4, LoRA +0.1; LoRA's loss goes up in epoch 1
Oct 18	Is LoRA's learning rate too high? → halve it	**failed**: −0.12 (Tab. 9)
Oct 18	Does LoRA need more capacity? → adapt the MLP too	+0.88 (Tab. 9)
Oct 19	Is the LP head the problem? → LP *vs.* random *vs.* prototype head	prototype +2.5; random ties it (Tab. 6)
Oct 19	Final recipe → prototype head; full FT 8 epochs; LoRA 3 *vs.* 6	val 76.22 and 71.90; LoRA 6 epochs **failed**: −0.46
Nov 2	Which method is best per parameter? → seven methods, one protocol	small changes in every block (Tab. 5)
Nov 5*	What limits us now? → ImageNet-21k-only backbones; larger images	backbone: no gain (Sec. 3.1); image size: +4.4 (Tab. 7)

Table 2. **How we iterated**, one question per round (holdout top-1 unless marked val; FT: fine-tuning). *After the deadline; no ranked submission uses it.

Metrics. Latency is the mean time of 100 forward passes of a random 224×224 input at batch size 1, after 10 warm-up passes, with bf16 autocast on an A100. Throughput is measured at batch size 256. For the same model, latency varies by up to about 0.5 ms between runs. GPU-hours come from the clock in Sec. 3.1. Peak VRAM is the most memory PyTorch allocated during training; the memory PyTorch reserved is 0.2–1.4 GB higher (20.1 GB for the final model). The appendix gives training-log excerpts with time and VRAM (Appendix A), all runs (Tab. 8), the hyperparameters (Tab. 11) and the hardware and software (Appendix C). All runs use one A100-SXM4-80GB, except one ablation batch whose accuracies we checked to be comparable (Appendix C).

4.2. How we iterated

We ran the study in rounds; each round asked one question and changed one setting (Tab. 2). The ideas that failed were useful too. When LoRA's training loss went up in its first epoch, we first blamed the learning rate. Halving it changed nothing, so we looked at the head next, which gave the largest gain of the project (Sec. 4.6). Training LoRA longer, which helped full fine-tuning, lowered its training loss but also its accuracy (Sec. 4.8).

4.3. Reproducing the reference baselines

Before changing anything, we checked our pipeline against the three reference results in the project description. Short 3-epoch runs match all three within 0.3 points (Tab. 3). So our data pipeline, our evaluation and the way we count trainable

Method	Trainable	Reference	Ours
Zero-shot, initialized head	0%	43.7	43.95
Full fine-tuning	100%	70.8	70.88
LoRA ($r = 8$)	0.34%	68.3	68.57

Table 3. **Reproducing the reference results** from the project description (top-1, %). Ours: holdout accuracy of 3-epoch runs (0.40–0.45 GPU-h) with the linear-probe head; for zero-shot, the prototype head without training.

parameters agree with the reference.

4.4. Main results

Table 4 gives all four required metrics for the key models. Our final full fine-tuning model reaches **76.22%** on the CodaBench validation set, 5.4 points above the reference. It uses 1.03 GPU-hours, 18.9 GB of memory and the unchanged ViT-B/16 architecture. Changing one setting at a time (Fig. 5 in the appendix), flip TTA adds 0.27 points (it changes 4.6% of the predictions and doubles latency), training on all 45 images per class adds 0.26 points (close to the noise, but in the expected direction), and replacing the LP head with the prototype head adds **3.12 points**. Our final LoRA model, on all linear layers with a prototype head, reaches **71.90%** (+3.6 over the reference LoRA). It trains 1.375% of the backbone, and after merging it has the same latency as the base model. The two final models were not trained the same way: the LoRA model keeps the holdout split and uses no TTA. For a fair comparison, under the same settings (3 epochs, prototype head, holdout) LoRA reaches 98% of the full fine-tuning accuracy (71.98% *vs*. 73.38%). On the hidden test set (Phase 2), the two final models reach **76.09%** and **71.67%**, within 0.25 points of their validation scores. Both models were fixed before the test set was released, and each used one of our two Phase 2 submissions.

4.5. Comparing adaptation methods

The project description lists several ways to adapt the backbone. To compare them fairly, we ran seven methods with the same protocol: the prototype head (Sec. 3.3), 3 epochs, the same holdout split and the same GPU (Tab. 5 and Fig. 2). All efficient methods use learning rate 10^{-3}; we did not tune it for each method.

Where to train. Small changes in every block give much more accuracy per parameter than fine-tuning the top blocks. BitFit trains 0.12% of the backbone and beats fine-tuning the last block (8.3%) by 1.8 points; LoRA (1.375%) matches the last four blocks (33%), and the Adapter (1.387%) beats the last two (16.5%). Each extra top block helps less: the first block adds 6.0 points over the linear probe, the second 3.0, and each further pair about 1; six blocks come within 0.5 points of full fine-tuning. A likely reason is that frozen lower blocks cannot change their features for the new classes, while small changes in every block reach all of them. Among the efficient methods, LoRA is the most accurate (71.98%), followed by the Adapter (71.54%), AdaptFormer (71.17%) and BitFit (69.81%). But LoRA is only 0.44 points ahead of the Adapter, close to our noise level; its clear advantage is that it can be merged. The serial Adapter and the parallel AdaptFormer differ by 0.37 points, and they also use different scales (1.0 *vs*. 0.1), so we cannot say which placement is better. None of these methods reaches full fine-tuning (73.38%), although the project description expects parameter-efficient methods to do better on small data. One possible reason: each class has few images, but the whole training set has 450,000, which is enough to fine-tune the full backbone.

Training memory. Training memory depends on how far back the gradients have to go, not on how many parameters are trained (Fig. 2, right). Partial fine-tuning adds about 1.5 GB per trained block (4.1, 7.0 and 10.0 GB for $K = 2, 4, 6$); for all 12 blocks this gives 18.8 GB, the same as full fine-tuning (18.9 GB). BitFit and the two adapters train at most 1.4% of the backbone but need 12.7–12.8 GB, more than fine-tuning half the network, because their gradients must reach the first block. They still use a third less memory than full fine-tuning and train 24–32% faster (1,407–1,509 *vs*. 1,139 img/s), since frozen weight matrices need no weight gradients. LoRA is the exception (19.9 GB, 888 img/s; Sec. 4.7). At inference, only the two adapters cost extra: 9.74 and 10.23 ms at batch size 1, against 7.33 ms for the plain model.

4.6. Ablation: starting the head

Table 6 compares the three head starts in four settings. In all four, the LP head is clearly the worst, 2.1–3.1 points below the prototype head. For LoRA, the random and prototype heads end up level (71.95% *vs*. 72.00%), but the prototype head learns faster: it is 5 points ahead after the first epoch (63.3% *vs*. 58.4%). Figure 3 shows that the LP head is 18 points better before training and worse after it.

Why does the LP head hurt? The probe was trained on *the same images that are then used for fine-tuning*, so it already fits them well. Over the first 100 steps of the LoRA runs, the mean training loss is 2.29 with the LP head, against 4.39 with the prototype head and 9.28 with a random head. So the backbone gets only a small training signal, mostly from the few images the probe gets wrong. When the features start to change, the trained head no longer matches them: in the LoRA run with the LP head, the training loss goes up from 2.22 to 2.69 during the first epoch, before any image is seen twice. Figure 4 shows the same for full fine-tuning: with the same settings, the LP model reaches a lower training loss but a much lower validation accuracy, so it memorizes

Method	Head init	Ep.	Trainable params		Top-1 accuracy (%)			Inference		Training	
			Backbone	Head	Holdout	Val	Test	ms (bs 1)	img/s	GPU-h	GB
Zero-shot	prototype	0	0	0	43.95	–	–	7.55	3,832	0.11	3.9
Linear probe	LP	–	0	7.69M	62.07	–	–	–	–	≈0.11	–
Full fine-tuning	LP	8	85.80M (100%)	7.69M	72.31	72.57	–	7.35	3,864	1.03	18.9
Full fine-tuning (final)†	prototype	8	85.80M (100%)	7.69M	–	**76.22**	**76.09**	14.95‡	1,919‡	1.03	18.9
LoRA, qkv only	LP	3	0.29M (0.344%)	7.69M	68.57	–	–	7.22	3,817	0.40	13.5
AdaptFormer	LP	3	1.19M (1.387%)	7.69M	69.05	–	–	9.89	3,461	0.39	12.8
LoRA, all linear (final)	prototype	3	1.18M (1.375%)	7.69M	**71.98**	**71.90**	**71.67**	7.61	3,869	0.54	19.9

Table 4. **Main results.** Holdout: 2 images per class held out from the training set; Val/Test: CodaBench validation/test (Phase 2). Backbone trainable parameters are also given as a share of the 85.8M backbone; the 10,000-class head (7.69M) is trained in every method except zero-shot. Latency, throughput, GPU-hours and peak VRAM are measured on one A100-SXM4-80GB; LoRA is measured after merging. †Trained on all 45 images per class (no holdout); all other rows, including the final LoRA model, are trained on 43 images per class without TTA. ‡With flip TTA (two forward passes); without TTA the model is the same as in the row above (7.35 ms).

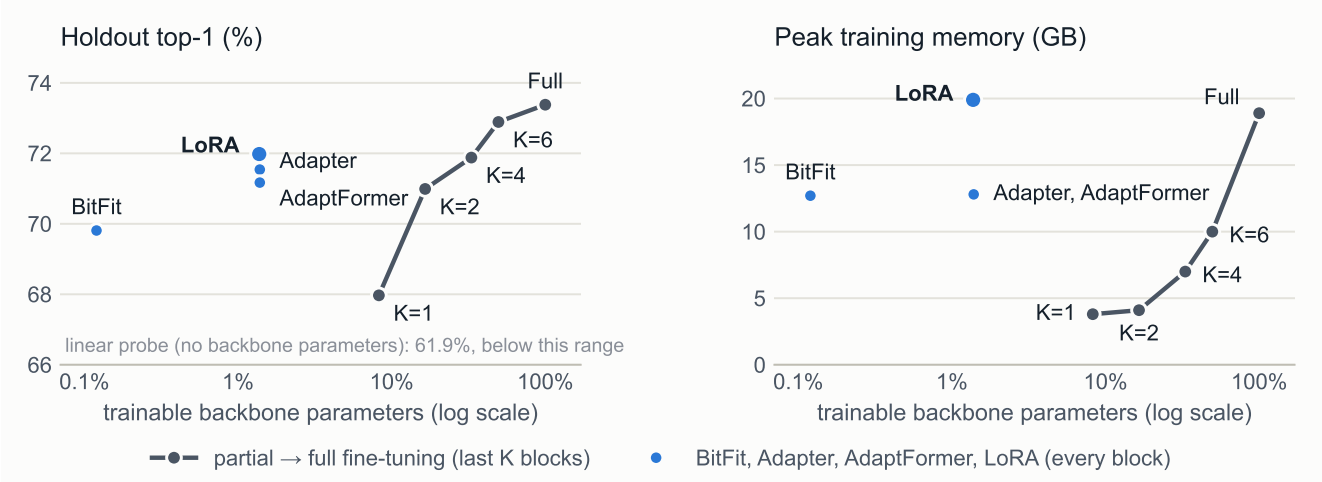


Figure 2. **Holdout accuracy (left) and peak training memory (right) against the share of trainable backbone parameters** (log scale) for the runs in Tab. 5. Methods that change every block (blue) get more accuracy per parameter than fine-tuning the last K blocks (gray), but need more memory, because their gradients go back to the first block. The linear probe (61.9%, no backbone parameters) is below the left panel; the memory of $K = 1$ is the feature-extraction peak.

more and generalizes less. The same holds for every pair in Tab. 6; for AdaptFormer, the LP run ends at a training loss of 1.86 *vs*. 1.98 and a holdout accuracy of 69.05% *vs*. 71.17%. The random head is our control: it knows nothing about the classes, yet it does as well as the prototype head. So the prototype head is not special; what hurts is a head that was trained on the training images. LP-FT [12] uses the probe so that a random head does not distort the pretrained features, mainly to keep accuracy on new domains. In our task the features have to change, and a random head did no harm.

4.7. Ablation: where to put PEFT capacity

Table 9 in the appendix compares PEFT variants with the LP head for 3 epochs. Capacity mattered more than the learning rate. Halving the LoRA learning rate changed nothing (−0.12, within noise). Adding LoRA to all four linear layers instead of only `qkv`, so that the MLP is adapted too, added 0.88 points; the MLP holds two thirds of the weights in each block. This is still far below the 5% that the project description gives as an example of a small set of parameters. Merging matters for deployment: AdaptFormer is close in accuracy, but its side branches cannot be merged, so batch-1 latency rises from about 7.3 ms to 9.89 ms (+35%). LoRA is not cheaper to train here. LoRA on all linear layers trains *slower* than full fine-tuning (885 *vs*. 1,131 img/s) and needs slightly more memory (19.9 *vs*. 18.9 GB). To train its A matrices, LoRA must store the input of every linear layer, as full fine-tuning does. On top of that come the activations of its 96 extra small matrix products, which probably outweigh the about 1 GB of weight gradients and optimizer state that full fine-tuning adds. LoRA saves trainable parameters, opti-

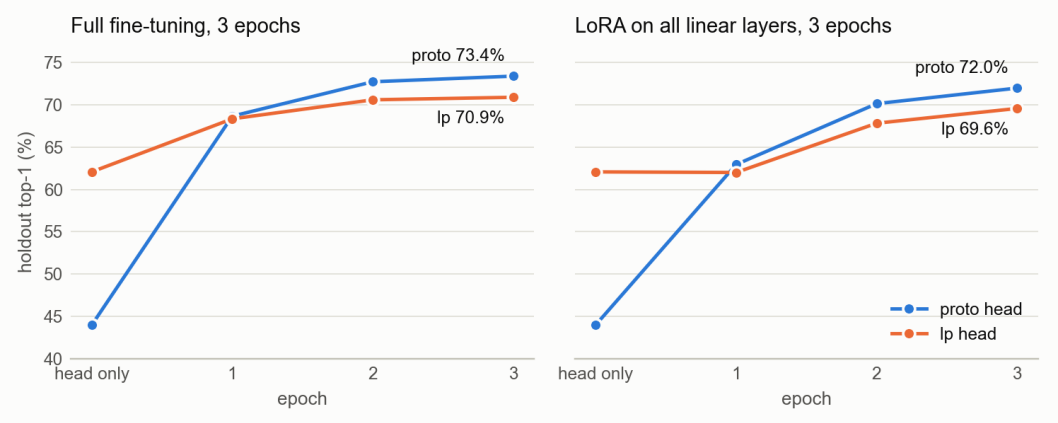


Figure 3. **Holdout accuracy during training for the LP and prototype heads** (3 epochs; left: full fine-tuning, right: LoRA on all linear layers). "head only" is the accuracy of the starting head before any training. The LP head starts 18 points higher but ends about 2.5 points lower. Points are the holdout checks after each epoch; for LoRA with the LP head, the final check after merging gives 69.45 (Tab. 6) instead of 69.56.

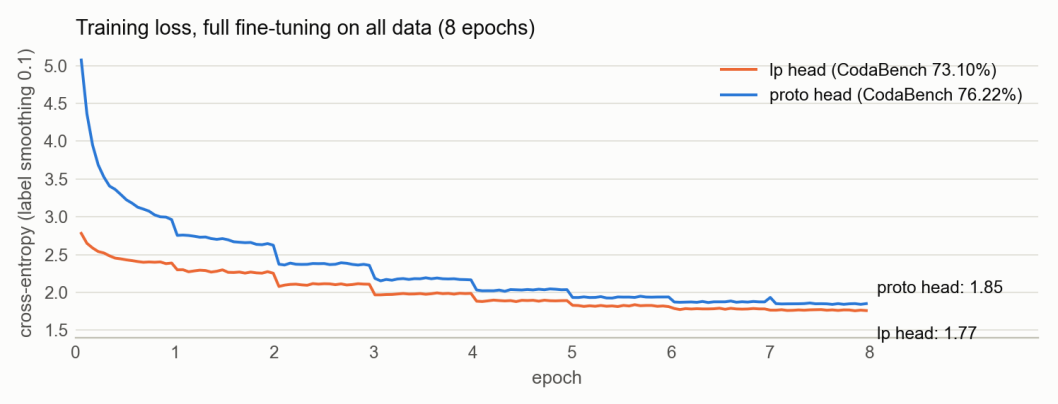


Figure 4. **Training loss of full fine-tuning on all data (8 epochs) with the LP and prototype heads**; nothing else differs. The prototype model has a *higher* training loss (1.85 *vs*. 1.77 in the last epoch) but scores 3.12 points higher on the validation set. The loss drops at each epoch boundary and stays flat within an epoch, a sign that the model memorizes the 45 images per class.

mizer state and checkpoint size (35.5 MB, of which 30.8 MB is the head, *vs*. 374 MB). The adapters and BitFit, in contrast, train faster and with a third less memory than full fine-tuning (Sec. 4.5).

4.8. Ablation: training length

Full fine-tuning and LoRA react to longer training in opposite ways (Tab. 10 in the appendix). Full fine-tuning keeps getting better: going from 3 to 8 epochs adds 2.21 points for the final recipe. LoRA stops improving after about 3 epochs, and with the prototype head 6 epochs are worse than 3, although its training loss keeps falling (1.95 to 1.70). We think the head is the cause: it holds 7.69M of LoRA's 8.87M trainable parameters. Compared with full fine-tuning, our LoRA recipe trains the head with a 10× higher learning rate, a 500× smaller weight decay and no drop path, so with more epochs it mostly memorizes the 43 images per class. A 6-epoch LoRA run with a head learning rate of 10^{-4} would test this; we have not run it yet. So our final LoRA model uses 3 epochs (0.54 GPU-hours) and our final full fine-tuning model uses 8 epochs (1.03 GPU-hours), both within the 1.2-hour budget.

4.9. After the deadline: input resolution

After the deadline, the gap to the top teams on the leaderboard made us ask what still limits our models. We kept the backbone and the protocol of Tab. 5 and changed only the image size. timm interpolates the 14×14 position embeddings to the patch grid of the input (18×18 at 288 pixels), so no parameter is added (Tab. 7). Testing our 224-pixel model at 288 pixels adds 2.1 points with no retraining. FixRes [21] explains this: random-resized crops make objects look larger

Prototype head, 3 epochs	Params	Holdout	img/s	GB	ms
Linear probe	0	61.94	–	3.2	6.96
Partial, $K = 1$	8.26%	67.97	1,706	3.8*	7.92
Partial, $K = 2$	16.5%	70.99	1,655	4.1	7.41
Partial, $K = 4^{\dagger}$	33.0%	71.88	1,611	7.0	7.41
Partial, $K = 6^{\dagger}$	49.6%	72.89	1,650	10.0	7.42
Full fine-tuning	100%	**73.38**	1,139	18.9	7.78
BitFit	0.120%	69.81	1,407	12.7	7.33
Adapter	1.387%	71.54	1,509	12.8	9.74
AdaptFormer	1.387%	71.17	1,436	12.8	10.23
LoRA, all linear	1.375%	**71.98**	888	19.9	7.61

Table 5. **Adaptation methods under one protocol** (holdout top-1, %; A100). Params: trainable backbone parameters. img/s: training throughput; GB: peak training memory; ms: batch-1 latency, LoRA merged. *Peak during feature extraction; training used less. †Full fine-tuning recipe (Sec. 3.1); $K = 1, 2$ use learning rate $3{\times}10^{-4}$ without layer decay or drop path. We re-ran the linear probe for this comparison (62.07 in Tab. 4).

Head init	Head alone	LoRA all 3 ep holdout	AdaptF. 3 ep holdout	Full FT 3 ep holdout	Full FT 8 ep, TTA val
Random	chance	71.95*	–	–	–
Prototype	44.0	**72.00***	**71.17**	**73.38**	**76.22**
Linear probe (LP)	62.1	69.45	69.05	70.88	73.10

Table 6. **Head initialization** (top-1, %). In each column only the head initialization changes. Head alone: accuracy of the starting head before training. AdaptF.: AdaptFormer. Full FT 8 ep: all 45 images per class, flip TTA, CodaBench validation. *RTX PRO 6000; repeating the prototype run on the A100 gives 71.98.

	Test size (pixels)					
Trained at	224	256	288	320	352	384
224 pixels	73.38	75.21	**75.50**	75.28	74.33	72.94
288 pixels	73.10	75.74	77.23	**77.78**	77.63	77.41

Table 7. **Input resolution** (after the deadline; full fine-tuning, prototype head, 3 epochs, holdout top-1, %; runs `1019_1120_1_full` and `1105_1031_1_full_288`). Position embeddings are interpolated to the patch grid of each size. Bold: best test size for each model.

during training, so slightly larger test images match them better. Training at 288 pixels adds another 2.3 points, and the best test size moves up to 320 pixels: 77.78%, 4.4 points above the 224-pixel model. This 3-epoch run fits the budget: 0.68 GPU-hours and 30.2 GB, against 0.48 GPU-hours and 18.9 GB at 224 pixels, because a 288-pixel image has 325 tokens instead of 197. We also trained the full 8-epoch recipe at 288 pixels on all images; it takes 1.61 GPU-hours, over the budget, and has no test score, so it is not one of our results. Input resolution gave the largest gain we found, but we found it too late.

5. Conclusion

We adapted ViT-B/16 to a 10,000-class fine-grained dataset with 45 images per class, with a budget of 96 GB·h per run. The biggest gain came from how we started the new head, not from the adaptation method. Starting the head from class prototypes instead of a linear probe trained on the training features added 2.1–3.1 points for full fine-tuning, LoRA and AdaptFormer. With this head, full fine-tuning reaches 76.22% on validation and 76.09% on test in 1.03 GPU-hours, and LoRA on all linear layers reaches 71.90% and 71.67% with 1.375% of the backbone and no extra inference cost after merging. In our comparison of seven methods, small changes in every block gave more accuracy per parameter than fine-tuning whole top blocks, with 12–69× fewer parameters. LoRA worked best on all linear layers and with a short schedule of about 3 epochs, and unlike the two adapters it can be merged. Training memory depends on how far back the gradients go: fine-tuning only the top blocks is the cheapest to train, while spreading a few parameters over all blocks gives the most accuracy per parameter.

Lessons from the process. First, failed ideas helped: halving LoRA's learning rate changed nothing, which pointed us from the optimizer to the head, and training LoRA longer lowered its training loss but also its accuracy, which showed that memorization, not underfitting, limits it. Second, a good start can be misleading: the linear-probe head starts 18 points ahead and ends 2.5 points behind, so we judged every choice on held-out data after training, never on the training loss or the starting accuracy. Third, we studied the adaptation method early and the input late: after the deadline, larger input images added 4.4 holdout points (Sec. 4.9), more than any single adaptation choice, while the recommended backbone did as well as the ImageNet-21k-only models (Sec. 3.1).

Limitations and next steps. Each setting was trained once (seed 0), and the holdout noise is about ±0.3 points, so small differences, such as the 0.26-point gain from two extra images per class, are uncertain. Our explanation of why the linear-probe head hurts fits the loss curves and the random-head control, but it is not a full proof. The efficient methods share one learning rate that we did not tune per method, and we only used LoRA rank 8. Partial fine-tuning with $K = 4, 6$ uses a different recipe from $K = 1, 2$, and the Adapter and AdaptFormer also differ in scale, so these comparisons change more than one thing at a time. We did not try prompt-based methods such as VPT [9] or prefix tuning [13]. Next, we would trade epochs for image size within the budget, for example 6 epochs at 224 pixels and 2 at 288 (about 1.2 GPU-hours by our estimate), and test whether a lower head learning rate lets LoRA train longer.

References

[1] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[2] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. 2, 3

[3] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting vision transformers for scalable visual recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2, 3

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009. 2

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. 1, 2

[6] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019. 1, 2, 3

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2, 3

[8] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, 2016. 2

[9] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision (ECCV)*, 2022. 2, 8

[10] Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 2

[11] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012. 4

[12] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2, 4, 6

[13] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021. 8

[14] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. 2, 3

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 2

[16] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022. 2

[17] Hang Qi, Matthew Brown, and David G. Lowe. Low-shot learning with imprinted weights. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 1, 2, 4

[18] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2, 4

[19] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research (TMLR)*, 2022. 1, 2

[20] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 2, 3

[21] Hugo Touvron, Andrea Vedaldi, Matthijs Douze, and Hervé Jégou. Fixing the train-test resolution discrepancy. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 7

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[23] Ross Wightman. PyTorch image models. https://github.com/huggingface/pytorch-image-models, 2019. 2

[24] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 2

A. Training-log excerpts

The project asks us to back up training time and VRAM with training logs. Below are parts of `train_log.txt` for our two final models; the full logs (every 100 steps) are available on request. Dates are removed from the timestamps, Google Drive paths are shortened, and long lines are wrapped. GPU-hours include head initialization (feature extraction), training and per-epoch holdout evaluation (Sec. 3.1).

Final full fine-tuning model (prototype head, all 45 images per class, 8 epochs, flip TTA at prediction):

```
========== 1019_1319_1_full ==========
[13:32:14] full: lr 0.0001, head lr 0.0001, wd 0.05, layer decay 0.75, epochs 8, holdout 0/class,
           head init proto, time budget 1.20 h
[13:32:14] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[13:38:32] Extracted features for 450,000 train / 0 holdout images in 6.1 min
[13:38:32] Head initialized from proto (6.2 min incl. feature extraction, counted in this run's GPU time)
[13:38:32] Trainable (excl. head): 85,798,656 = 100.000% of backbone | head 7,690,000 params (fine-tuned)
[13:38:32] Training up to 8 epochs x 1757 steps (batch 256x1, warmup 1757 steps)
[13:40:03] Speed 1146 img/s, 6.5 min/epoch
[13:46:13] == epoch 1: train loss 3.4158, 13.8 min used
[13:52:55] == epoch 2: train loss 2.6899, 20.5 min used
[13:59:51] == epoch 3: train loss 2.3729, 27.4 min used
[14:06:42] == epoch 4: train loss 2.1758, 34.2 min used
[14:13:40] == epoch 5: train loss 2.0321, 41.1 min used
[14:20:31] == epoch 6: train loss 1.9367, 47.9 min used
[14:27:32] == epoch 7: train loss 1.8741, 54.9 min used
[14:34:26] == epoch 8: train loss 1.8491, 61.8 min used
[14:34:29] Training done: 14056 steps, 1.03 GPU-hours (budget 1.20 h)
[14:34:29] Peak VRAM: 18.9 GB allocated, 20.1 GB reserved by PyTorch (GPU has 80 GB)
[14:34:32] Inference latency (batch 1): 14.95 ms/img | throughput (batch 256): 1919 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, with flip TTA
[14:35:29] [val] 50,000 predictions over 9,992 classes (max 13 per class) ->
           .../submission_val_1019_1319_1_full.zip
```

Final LoRA model (LoRA on all linear layers, prototype head, 3 epochs, 2 images per class held out, no TTA):

```
========== 1019_1319_3_lora ==========
[14:57:51] lora: lr 0.001, head lr 0.001, wd 0.0001, layer decay 1.0, epochs 3, holdout 2/class, head init proto,
           time budget 1.20 h
[14:57:51] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[15:06:07] Extracted features for 430,000 train / 20,000 holdout images in 7.2 min
[15:06:08] Prototype head (zero-shot, 0 trainable params): holdout top-1 44.01%
[15:06:08] Head initialized from proto (7.2 min incl. feature extraction, counted in this run's GPU time)
[15:07:15] Trainable (excl. head): 1,179,648 = 1.375% of backbone | head 7,690,000 params (fine-tuned)
[15:07:15] Training up to 3 epochs x 1679 steps (batch 256x1, warmup 503 steps)
[15:07:53] Speed 888 img/s, 8.1 min/epoch
[15:15:44] == epoch 1: train loss 3.2174, holdout top-1 62.94%, 15.7 min used
[15:24:12] == epoch 2: train loss 2.4125, holdout top-1 70.12%, 24.2 min used
[15:32:40] == epoch 3: train loss 1.9496, holdout top-1 71.97%, 32.6 min used
[15:32:41] Training done: 5037 steps, 0.54 GPU-hours (budget 1.20 h)
[15:32:41] Peak VRAM: 19.9 GB allocated, 21.3 GB reserved by PyTorch (GPU has 80 GB)
[15:33:00] Final holdout: top-1 71.98%, top-5 89.07%
[15:33:03] Inference latency (batch 1): 7.61 ms/img | throughput (batch 256): 3869 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, no flip TTA
[15:33:55] [val] 50,000 predictions over 9,991 classes (max 15 per class) ->
           .../submission_val_1019_1319_3_lora.zip
```

B. All runs

Run	Method	Head	Ep.	Trainable	Holdout	Val	GPU-h	GB	ms
`0928_0636_1_linear`	Zero-shot	prototype	0	0	43.95	–	0.11	3.9	7.55
`0928_0636_3_full`	Full FT	LP	3	100%	70.88	–	0.45	18.9	7.31
`0928_0748_1_full`	Full FT	LP	8	100%	72.31	72.57/72.84‡	1.03	18.9	7.35
`1018_2306_1_full`	Full FT, all data	LP	8	100%	–	73.10‡	1.03	18.9	14.92‡
`1019_1120_1_full`	Full FT	prototype	3	100%	73.38	–	0.48	18.9	7.78
`1019_1319_2_full`	Full FT, all data	prototype	3	100%	–	74.01‡	0.44	18.9	15.24‡
`1019_1319_1_full`	**Full FT, all data (final)**	prototype	8	100%	–	**76.22**‡	1.03	18.9	14.95‡
`0928_0636_2_lora`	LoRA qkv	LP	3	0.344%	68.57	–	0.40	13.5	7.22
`0928_0748_2_lora`	LoRA qkv, head lr 2×10^{-4}	LP	10	0.344%	68.69	–	1.06	13.5	7.53
`1018_1239_1_lora`	LoRA qkv, lr 5×10^{-4}	LP	3	0.344%	68.45	–	0.41	13.5	7.15
`1018_1239_3_adaptformer`	AdaptFormer	LP	3	1.387%	69.05	–	0.39	12.8	9.89
`1018_1239_2_lora`	LoRA all	LP	3	1.375%	69.45	–	0.55	19.9	7.03
`1018_2306_2_lora`	LoRA all	LP	6	1.375%	69.87	–	0.97	19.9	7.31
`1019_0953_1_lora`*	LoRA all	random	3	1.375%	71.95	–	0.37*	19.9	1.81*
`1019_0953_2_lora`*	LoRA all	prototype	3	1.375%	72.00	–	0.44*	19.9	1.82*
`1019_1120_2_lora`	LoRA all	prototype	6	1.375%	71.52	–	0.97	19.9	7.76
`1019_1319_3_lora`	**LoRA all (final PEFT)**	prototype	3	1.375%	**71.98**	**71.90**	0.54	19.9	7.61
`1102_1149_1_linear_proto`	Zero-shot (re-run)	prototype	0	0	43.95	–	0.10	3.8	7.32
`1102_1149_2_linear_lp`	Linear probe	LP	–	0	61.94	–	0.10	3.2	6.96
`1102_1942_1_partial_k1`	Partial FT, $K = 1$	prototype	3	8.263%	67.97	–	0.35	3.8	7.92
`1102_1942_2_partial_k2`	Partial FT, $K = 2$	prototype	3	16.524%	70.99	–	0.35	4.1	7.41
`1102_1942_3_partial_k4`	Partial FT, $K = 4$	prototype	3	33.046%	71.88	–	0.35	7.0	7.41
`1102_1942_4_partial_k6`	Partial FT, $K = 6$	prototype	3	49.568%	72.89	–	0.35	10.0	7.42
`1102_2023_1_bitfit`	BitFit	prototype	3	0.120%	69.81	–	0.39	12.7	7.33
`1102_2017_1_adapter`	Adapter	prototype	3	1.387%	71.54	–	0.37	12.8	9.74
`1102_2212_1_adaptformer`	AdaptFormer	prototype	3	1.387%	71.17	–	0.39	12.8	10.23

Table 8. **All 26 runs.** IDs encode the batch start time (month, day, hour, minute) and the position in the batch; the last block is the method comparison (Sec. 4.5). LoRA all: LoRA on all linear layers. Trainable: backbone parameters without the 7.69M head. GB: peak allocated training memory ($K = 1$: feature extraction). ms: batch-1 latency. ‡Flip TTA. *RTX PRO 6000 Blackwell; all other runs on an A100-SXM4-80GB (speed and latency are not comparable). The LP head (62.07%) is computed inside every LP run; `1102_1149_2` re-ran it alone.

Runs after the deadline (Sec. 4.9). None of them is used for a ranked result. Batch `1105_0141` ran zero-shot and the linear probe with the ImageNet-21k-only checkpoints `augreg_in21k` and `orig_in21k` (Sec. 3.1), and evaluated `1019_1120_1_full` at six test sizes. `1105_1031_1_full_288` trains 3 epochs at 288 pixels on the holdout split (0.68 GPU-h, 30.2 GB). `1105_1032_1_full_288` trains the final recipe at 288 pixels on all images (1.61 GPU-h, 30.2 GB); it was not submitted, so it has no test score.

C. Hardware, software and the path to the final model

All runs use one NVIDIA A100-SXM4-80GB with 12 CPU cores, except one ablation batch (random and prototype heads for LoRA in Tab. 6), which ran on an RTX PRO 6000 Blackwell. Repeating its prototype run on the A100 gave 71.98% *vs.* 72.00%, so accuracies are comparable across the two GPUs; speeds and latencies are not. Software, as logged from batch `1019_1120` on, which covers all final models and the method comparison: PyTorch 2.11.0 (CUDA 13.0), timm 1.0.30 and PEFT 0.21.1. Earlier runs used the then-current releases, which were not logged. Figure 5 shows the validation accuracy of the final full fine-tuning recipe as we changed one setting at a time.

D. Ablation tables

Tables 9 and 10 give the numbers behind Secs. 4.7 and 4.8.

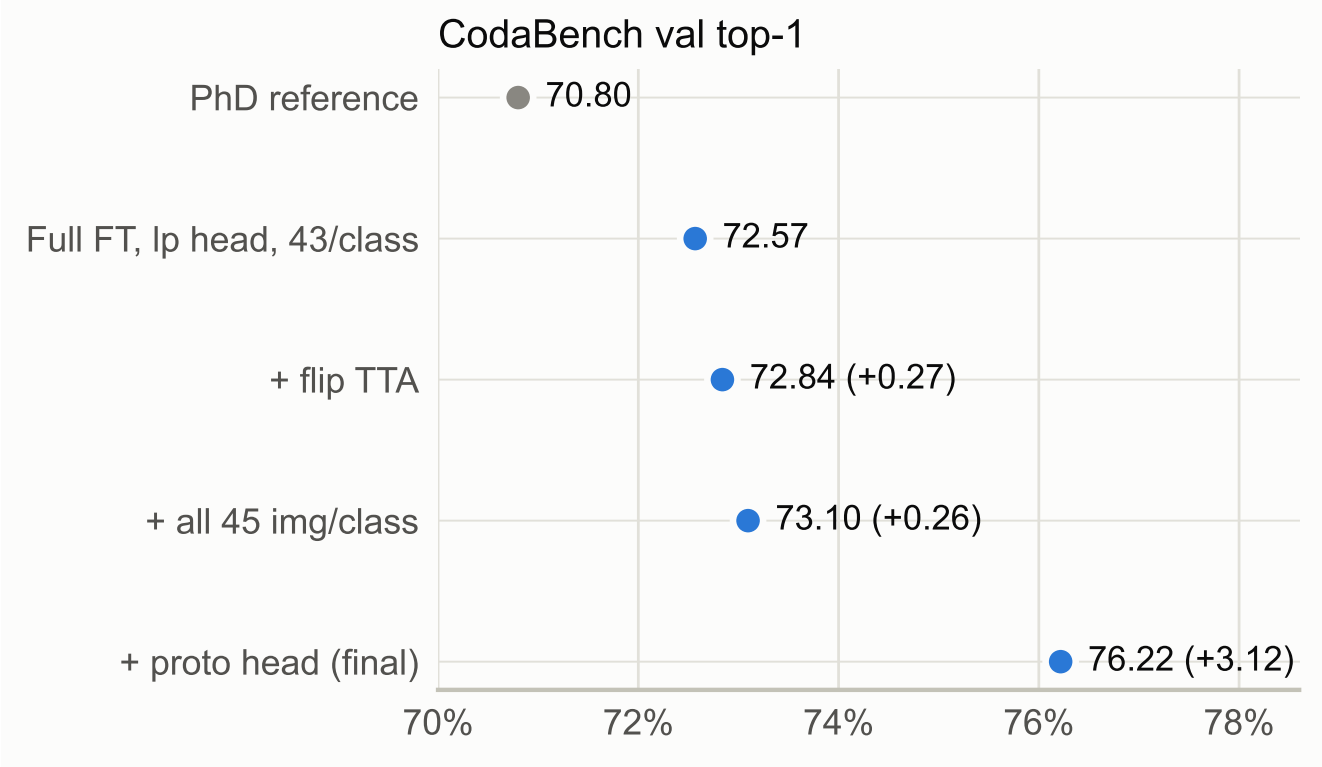


Figure 5. **CodaBench validation accuracy as we change one setting at a time** (all runs: full fine-tuning, 8 epochs). The gray dot is the reference. Positions encode values, so the axis starts at 70%.

LP head, 3 epochs	Params	Holdout	img/s	GB	ms
LoRA qkv, lr 10^{-3}	0.344%	68.57	1,341	13.5	7.22
LoRA qkv, lr 5×10^{-4}	0.344%	68.45	1,353	13.5	7.15
LoRA all linear	1.375%	**69.45**	885	19.9	7.03
AdaptFormer	1.387%	69.05	1,437	12.8	9.89
Full fine-tuning	100%	70.88	1,131	18.9	7.31

Table 9. **PEFT design.** Params: trainable backbone parameters (head excluded). img/s: training throughput; GB: peak training memory; ms: batch-1 latency after merging (A100, bf16).

Setting	Short (ep)	Long (ep)	Δ
Full FT, LP	70.88 (3)	72.31 (8)	+1.43
Full FT, prototype, final[‡]	74.01 (3)	76.22 (8)	+2.21
LoRA qkv, LP[†]	68.57 (3)	68.69 (10)	+0.12
LoRA all, LP	69.45 (3)	69.87 (6)	+0.42
LoRA all, prototype	71.98 (3)	71.52 (6)	−0.46

Table 10. **Training length** (holdout top-1, %; LoRA all: LoRA on all linear layers). [‡]All 45 images per class with flip TTA, CodaBench validation. [†]The 10-epoch run also lowered the head learning rate to 2×10^{-4}.

E. Hyperparameters

Methods added for Sec. 4.5 train for 3 epochs with the settings above, except: partial fine-tuning uses learning rate 3×10^{-4} and weight decay 0.05 for $K = 1, 2$ and the full fine-tuning settings for $K = 4, 6$ (drop path only in trained blocks); BitFit uses learning rate 10^{-3} and no weight decay; the Adapter is AdaptFormer's bottleneck placed after the MLP, with scale 1.0.

	Full fine-tuning	LoRA	AdaptFormer	Linear-probe head
Trainable	all weights + head	A, B in qkv, proj, fc1, fc2 (all 12 blocks) + head	parallel MLP adapters + head	head only, on cached features
Method-specific	layer-wise lr decay 0.75, drop path 0.1	$r = 8$, $\alpha = 16$, no dropout, no layer decay, no drop path, merged after training	bottleneck 64, scale 0.1, dropout 0.1	initialized from the prototype head
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning rate	10^{-4} (head 10^{-4})	10^{-3} (head 10^{-3})	10^{-3} (head 10^{-3})	10^{-3}
Weight decay	0.05	10^{-4}	10^{-4}	10^{-4}
Schedule	linear warmup (1 epoch; 0.3 epochs for 3-epoch runs), then cosine decay to 0			cosine, no warmup
Epochs	3 or 8	3, 6 or 10	3	10
Batch size	256	256	256	4,096
Precision	bf16	bf16	bf16	fp32 head, fp16 features
Regularization	label smoothing 0.1, gradient clipping at norm 1.0			label smoothing 0.1
Augmentation	RandomResizedCrop (scale 0.25–1) + horizontal flip			none (center crops)

Table 11. **Hyperparameters.** Weight decay is not applied to biases, normalization parameters, the class token and position embeddings. Evaluation resizes to 248 pixels and takes a 224 center crop.